CS24301 — THEORY (3.0 CR) | 10-PAGE MASTER REVISION | BIT MESRA | NEP SCHEME

Compiler Design — 10-Page Master Quick Revision

High-Yield Formula Sheet, Decision Trees & Top 10 BIT Mesra PYQ Solutions

⚡ 10-Page Master Quick Revision — Compiler Design (CS24301)

📋 The Complete 47-Topic Syllabus Master Checklist

47 / 47 Syllabus Topics Complete

Module	Topic #	Topic Name	Status
M1: Lexical Analysis (7)	1	Introduction to Compilers and its Cousins (P-I-C-A-L)	✅ Complete
	2	Structure of a Compiler (Front End vs. Back End, 6 Phases)	✅ Complete
	3	Lexical Analyzer (Token, Lexeme, Pattern)	✅ Complete
	4	Input Buffering (Two-Buffer Scheme, Sentinels)	✅ Complete
	5	Specification of Tokens (Regular Expressions & Operators)	✅ Complete
	6	Recognition of Tokens (Transition Diagrams & DFAs)	✅ Complete
	7	DFA Directly from Regular Expressions (FLF Rules)	✅ Complete
M2: Syntax Analysis (10)	8	Introduction to Syntax Analysis & CFG Grammars	✅ Complete
	9	Grammar Rewriting (Left Recursion & Left Factoring)	✅ Complete
	10	Recursive Top-Down Parsers (Recursive Descent)	✅ Complete
	11	Non-Recursive Top-Down Parsers (Predictive Parsing)	✅ Complete
	12	Design of LL(1) Parser (FIRST & FOLLOW Inductive Rules)	✅ Complete
	13	Bottom-Up Parsers (Shift-Reduce Parsing & Handle Pruning)	✅ Complete
	14	Variants of LR Parsers (LR(0), SLR(1), CLR(1), LALR(1))	✅ Complete
	15	Handling Parsing Conflicts (Shift-Reduce & Reduce-Reduce)	✅ Complete
	16	Detection of Syntax Errors (Panic-Mode & Phrase-Level)	✅ Complete
	17	Reporting of Syntax Errors & Diagnostics	✅ Complete
M3: Semantic Analysis (9)	18	Introduction to Semantic Analysis & Static Checks	✅ Complete
	19	Syntax-Directed Definitions (SDD Semantic Rules)	✅ Complete
	20	Syntax-Directed Translation Schemes (SDTS Actions)	✅ Complete
	21	SDTS for Declaration Processing & Symbol Tables	✅ Complete
	22	Three Address Code (Quadruples, Triples, Indirect Triples)	✅ Complete
	23	Types of Attributes (Synthesized vs. Inherited)	✅ Complete
	24	Type Checking for Expressions (Equivalence & Coercion)	✅ Complete
	25	Intermediate Code Generation for Assignment Statements	✅ Complete
	26	Translation of Multi-Dimensional Array References	✅ Complete
M4: Intermediate & Runtime (9)	27	Complete Evaluation of Boolean Expressions	✅ Complete
	28	Partial Evaluation (Short-Circuit) of Boolean Expressions	✅ Complete
	29	Translation of Control Flow Constructs (`if-else`, `while`)	✅ Complete
	30	Resolution of Forward Jumps & Backpatching	✅ Complete
	31	Resolution of Backward Jumps in Loops	✅ Complete
	32	Translation of Function Calls (`param`, `call`)	✅ Complete

Module	Topic #	Topic Name	Status
	33	Translation of Function Returns & Caller Restoration	✅ Complete
	34	Memory Layout of Code and Data (Text, Static, Heap, Stack)	✅ Complete
	35	Activation Records (Stack Frame Anatomy: P-R-C-A-S-L-T)	✅ Complete
M5: Optimization & Code Gen (12)	36	Addresses of Code and Data in Assembly Code	✅ Complete
	37	Correlation of Assembly Code with Source Code	✅ Complete
	38	Construction of Basic Blocks (3 Leader Rules)	✅ Complete
	39	Control Flow Graph (CFG Nodes & Edges)	✅ Complete
	40	Machine-Independent Local Optimizations	✅ Complete
	41	Machine-Independent Global Optimizations	✅ Complete
	42	Unreachable Code Elimination	✅ Complete
	43	Constant Folding Optimization	✅ Complete
	44	Constant Propagation Optimization	✅ Complete
	45	Loop-Invariant Code Motion (Hoisting)	✅ Complete
	46	Common Subexpression Elimination (CSE)	✅ Complete
	47	Dead Code Elimination	✅ Complete

🧠 Master Conceptual Differences Flashcards

Concept Pair	Core Distinction & Memory Cue
Token vs. Lexeme vs. Pattern	`Token` = Abstract Category; `Lexeme` = Concrete characters in code; `Pattern` = Regex rule.
Compiler vs. Interpreter	`Compiler` = Translates entire program to binary upfront; `Interpreter` = Translates & executes line-by-line.
Syntax vs. Semantics	`Syntax` = Grammatical structure (Parse tree); `Semantics` = Logical meaning & type consistency.
SDD vs. SDTS	`SDD` = High-level attribute grammar + semantic rules; `SDTS` = Grammar with explicit action code `{...}` embedded.
Synthesized vs. Inherited	`Synthesized` = Computed from children (Bottom-up); `Inherited` = Passed from parent/siblings (Top-down).
Quadruple vs. Triple	`Quadruple` = Explicit `(op, arg1, arg2, result)`; `Triple` = Implicit index reference `(op, arg1, arg2)`.
Control Link vs. Access Link	`Control Link` = Points to dynamic caller frame; `Access Link` = Points to static lexical enclosing scope.
Constant Folding vs. Propagation	`Folding` = Evaluates `3 + 4 -> 7`; `Propagation` = Substitutes variable `x=7; y=x+2 -> y=7+2`.
Unreachable vs. Dead Code	`Unreachable` = Can never execute (no path); `Dead` = Executes, but result is never used.